APPENDIX

B

Standard Delay Format (SDF)

This appendix describes the standard delay annotation format and explains how backannotation is performed in simulation. The delay format describes cell delays and interconnect delays of a design netlist and is independent of the language the design may be described in, may it be VHDL or Verilog HDL, the two dominant standard hardware description languages.

While this chapter describes backannotation for simulation, backannotation for STA is a more simple and straightforward process in which the timing arcs in a DUA are annotated with the specified delays from the SDF.

B.1 What is it?

SDF stands for Standard Delay Format. It is an IEEE standard - IEEE Std 1497. It is an ASCII text file. It describes timing information and constraints. Its purpose is to serve as a textual timing exchange medium between various tools. It can also be used to describe timing data for tools that require it. Since it is an IEEE standard, timing information generated by one tool can be consumed by a number of other tools that support such a standard. The data is represented in a tool-independent and language-independent way and it includes specification of interconnect delays, device delays and timing checks.

Since SDF is an ASCII file, it is human-readable, though these files tend to be rather large for real designs. However, it is meant as an exchange medium between tools. Quite often when exchanging information, one could potentially run into a problem where a tool generates an SDF file but the other tool that reads SDF does not read the SDF properly. The tool reader could either generate an error or a warning reading the SDF or it might interpret the values in the SDF incorrectly. In that case, one may have to look into the file and see what went wrong. This chapter explains the basics of the SDF file and provides necessary and sufficient information to help understand and debug any annotation problems.

Figure B-1 shows a typical flow of how an SDF file is used. A timing calculator tool typically generates the timing information that is stored in an SDF file. This information is then backannotated into the design by the tool that reads the SDF. Note that the complete design information is not captured in an SDF file, but only the delay values are stored. For example, instance names and pin names of instances are captured in the SDF as they are necessary to specify instance-specific or pin-specific delays. Therefore, it is imperative that the same design be presented to both the SDF generation tool and the SDF reader tool.

One design can have multiple SDF files associated with it. One SDF file can be created for one design. In a hierarchical design, multiple SDFs may be created for each block in a hierarchy. During annotation, each SDF is ap-

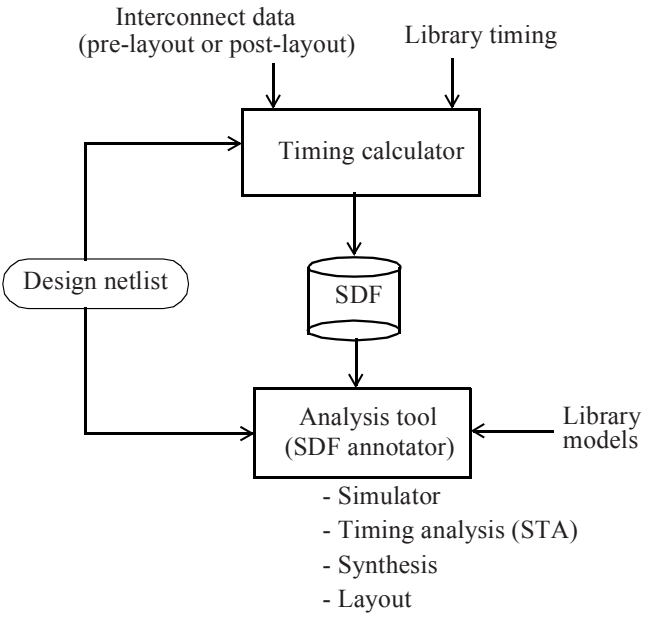

Figure B-1 *The SDF flow.*

plied to the appropriate hierarchical instance. Figure B-2 shows this figuratively.

An SDF file contains computed timing data for backannotation and for forward-annotation. More specifically, it contains:

i. Cell delays

ii. Pulse propagation

iii. Timing checks

iv. Interconnect delays

v. Timing environment

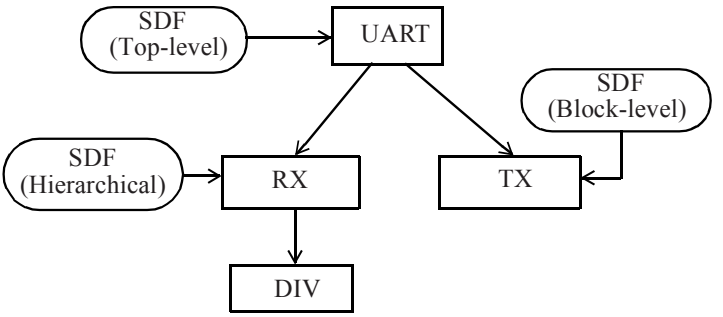

Figure B-2 *Multiple SDFs in a hierarchical design.*

Both pin-to-pin delay and distributed delay can be modeled for cell delays. Pin-to-pin delays are represented using the IOPATH construct. These constructs define input to output path delays for each cell. The COND construct can additionally be used to specify conditional pin-to-pin delays. State-dependent path delays can be specified using the COND construct as well. Distributed delay modeling is specified using the DEVICE construct.

The pulse propagation constructs, PATHPULSE and PATHPULSEPERCENT, can be used to specify the size of glitches that are allowed to propagate to the output of a cell using the pin-to-pin delay model.

The range of timing checks that can be specified in SDF includes:

i. Setup: SETUP, SETUPHOLD

ii. Hold: HOLD, SETUPHOLD

iii. Recovery: RECOVERY, RECREM

iv. Removal: REMOVAL, RECREM

v. Maximum skew: SKEW, BIDIRECTSKEW

vi. Minimum pulse width: WIDTH

vii. Minimum period: PERIOD

viii. No change: NOCHANGE

Conditions may be present on signals in timing checks. Negative values are allowed in timing checks, though tools that don't support negative values can choose to replace it with zero.

There are three styles of interconnect modeling that are supported in an SDF description. The INTERCONNECT construct is the most general and often used and can be used to specify point-to-point delay (from source to sink). Thus a single net can have multiple INTERCONNECT constructs. The PORT construct can be used to specify nets delays at only the load ports - it assumes that there is only one source for the net. The NETDELAY construct can be used to specify the delay of an entire net without regard to the sources or its sinks and therefore is the least specific way of specifying delays on a net.

The timing environment provides information under which the design operates. Such information includes the ARRIVAL, DEPARTURE, SLACK and WAVEFORM constructs. These constructs are mainly used for forward-annotation, such as for synthesis.

B.2 The Format

An SDF file contains a header section followed by one or more cells. Each cell represents a region or scope in a design. It can be a library primitive or a user-defined black box.

```
(DELAYFILE
 <header_section>
 (CELL
  <cell_section>
 )
 (CELL
  <cell_section>
 )
```

```
    ... <other cells>
)
```

The header section contains general information and does not affect the semantics of the SDF file, except for the hierarchy separator, timescale and the SDF version number. The hierarchy separator, DIVIDER, by default is the dot (`'.'`) character. It can be replaced with the `'/'` character by specifying:

```
(DIVIDER /)
```

If no timescale information is present in the header, the default is 1ns. Otherwise a timescale, TIMESCALE, can be explicitly specified using:

```
(TIMESCALE 10ps)
```

which says to multiply all delay values specified in the SDF file by 10ps.

The SDF version, SDFVERSION, is required and is used by the consumer of SDF to ensure that the file conforms to the specified SDF version. Other information that may be present in the header section, which is a general information category, includes date, program name, version and operating condition.

```
(DESIGN "BCM")
(DATE "Tuesday, May 24, 2004")
(PROGRAM "Star Galaxy Automation Inc., TimingTool")
(VERSION "V2004.1")
(VOLTAGE 1.65:1.65:1.65)
(PROCESS "1.000:1.000:1.000")
(TEMPERATURE 0.00:0.00:0.00)
```

Following the header section is a description of one or more cells. Each cell represents one or more instances (using wildcard) in the design. A cell may either be a library primitive or a hierarchical block.

```
(CELL
 (CELLTYPE <cell_type>)
 (INSTANCE <hierarchical_instance_name>)
 (DELAY
  <path_delay_section>
 )
 (TIMINGCHECK
  <timing_check_section>
 )
 (TIMINGENV
  <timing_environment_section>
 )
 (LABEL
  <label_section>
 )
)
. . . <other cells>
```

The order of cells is important as data is processed top to bottom. A later cell description may override timing information specified by an earlier cell description (usually it is not common to have timing information of the same cell instance defined twice). In addition, timing information can be annotated either as an absolute value or as an increment. If timing is incrementally applied, it adds the new value to the existing value; if the timing is absolute, it overwrites any previously specified timing information.

The cell instance can be a hierarchical instance name. The separator used for hierarchy separator must conform to the one specified in the header section. The cell instance name can optionally be the '*' character referring to a wildcard character, which means all cell instances of the specified type.

```
(CELL
 (CELLTYPE "NAND2")
 (INSTANCE *)
 // Refers to all instances of NAND2.
 . . .
```

There are four types of timing specifications that can be described in a cell:

i. DELAY: Used to describe delays.

ii. TIMINGCHECK: Used to describe timing checks.

iii. TIMINGENV: Used to describe the timing environment.

iv. LABEL: Declares timing model variables that can be used to describe delays.

Here are some examples.

```
// An absolute path delay specification:
(DELAY
  (ABSOLUTE
    (IOPATH A Y (0.147))
  )
)

// A setup and hold timing check specification:
(TIMINGCHECK
  (SETUPHOLD (posedge Q) (negedge CK) (0.448) (0.412))
)

// A timing constraint between two points:
(TIMINGENV
  (PATHCONSTRAINT UART/ENA UART/TX/CTRL (2.1) (1.5))
)

// A label that overrides the value of a Verilog HDL
// specparam:
(LABEL
  (ABSOLUTE
    (t$CLK$Q (0.480:0.512:0.578) (0.356:0.399:0.401))
    (tsetup$D$CLK (0.112))
  )
)
```

There are four types of DELAY timing specifications:

i. ABSOLUTE: Replaces existing delay values for cell instance during backannotation.

ii. INCREMENT: Adds the new delay data to any existing delay values of the cell instance.

iii. PATHPULSE: Specifies pulse propagation limit between an input and output of the design. This limit is used to decide whether to propagate a pulse appearing on the input to the output, or to be marked with an `'X'`, or to get filtered out.

iv. PATHPULSEPERCENT: This is exactly identical to PATHPULSE except that the values are described as percents.

Here are some examples.

```
// Absolute port delay:
(DELAY
  (ABSOLUTE
    (PORT UART.DIN (0.170))
    (PORT UART.RX.XMIT (0.645))
  )
)

// Adds IO path delay to existing delays of cell:
(DELAY
  (INCREMENT
    (IOPATH (negedge SE) Q (1.1:1.22:1.35))
  )
)

// Pathpulse delay:
(DELAY
  (PATHPULSE RN Q (3) (7))
)
// The ports RN and Q are input and output of the
```

```
// cell. The first value, 3, is the pulse rejection
// limit, called r-limit; it defines the narrowest pulse
// that can appear on output. Any pulse narrower than
// this is rejected, that is, it will not appear on
// output. The second value, 7, if present, is the
// error limit - also called e-limit. Any pulse smaller
// than e-limit causes the output to be an X.
// The e-limit must be greater than r-limit. See
// Figure B-3. When a pulse that is less than 3 (r-limit)
```

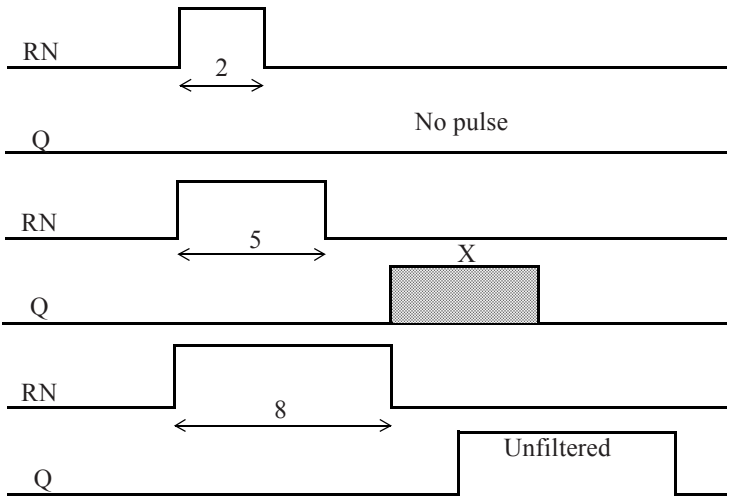

Figure B-3 *Error limit and rejection limit.*

```
// occurs, the pulse does not propagate to the output.
// When the pulse width is between the 3 (r-limit) and
// 7 (e-limit), the output is an X. When the pulse width
// is larger than 7 (e-limit), pulse propagates to output
// without any filtering.
```

```
// Pathpulsepercent delay type:
(DELAY
  (PATHPULSEPERCENT CIN SUM (30) (50))
)
// The r-limit is specified as 30% of the delay time from
// CIN to SUM and the e-limit is specified as 50% of
// this delay.
```

There are eight types of delay definitions that can be described with either ABSOLUTE or INCREMENT:

i. IOPATH: Input-output path delays.

ii. RETAIN: Retain definition. Specifies the time for which an output shall retain its previous value after a change on its related input port.

iii. COND: Conditional path delay. Can be used to specify state-dependent input-to-output path delays.

iv. CONDELSE: Default path delay. Specifies default value to use for conditional paths.

v. PORT: Port delays. Specifies interconnect delays that are modeled as delay at input ports.

vi. INTERCONNECT: Interconnect delays. Specifies the propagation delay across a net from a source to its sink.

vii. NETDELAY: Net delays. Specifies the propagation delay from all sources to all sinks of a net.

viii. DEVICE: Device delay. Primarily used to describe a distributed timing model. Specifies propagation delay of all paths through a cell to the output port.

Here are some examples.

```
// IO path delay between posedge of CK and Q:
(DELAY
```

```
  (ABSOLUTE
    (IOPATH (posedge CK) Q (2) (3))
  )
)
// 2 is the propagation rise delay and 3 is the
// propagation fall delay.

// Retain delay in an IO path:
(DELAY
  (ABSOLUTE
    (IOPATH A Y
      (RETAIN (0.05:0.05:0.05) (0.04:0.04:0.04))
      (0.101:0.101:0.101) (0.09:0.09:0.09))
  )
)
// Y shall retain its previous value for 50ps (40ps for
// a low value) after a change of value on input A.
// 50ps is the retain high value, 40ps is the retain
// low value, 101ps is the propagate rise delay and
// 90ps is the propagate fall delay. See Figure B-4.
```

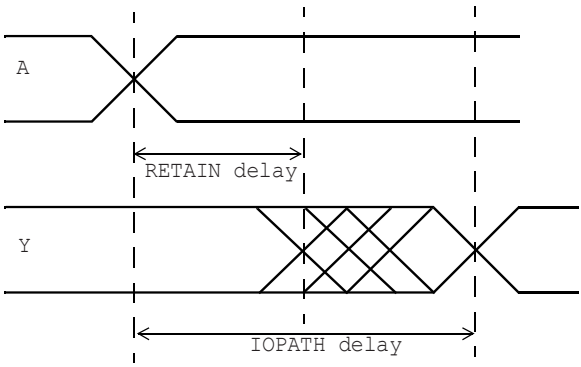

Figure B-4 *RETAIN delay.*

```
// Conditional path delay:
(DELAY
  (ABSOLUTE
    (COND SE == 1'b1 (IOPATH (posedge CK) Q (0.661)))
  )
)

// Default conditional path delay:
(DELAY
  (ABSOLUTE
    (CONDELSE (IOPATH ADDR[7] COUNT[0] (0.870) (0.766)))
  )
)

// Port delay on input FRM_CNT[0]:
(DELAY
  (ABSOLUTE
    (PORT UART/RX/FRM_CNT[0] (0.439))
  )
)

// Interconnect delay:
(DELAY
  (ABSOLUTE
    (INTERCONNECT O1/Y O2/B (0.209:0.209:0.209))
  )
)

// Net delay:
(DELAY
  (ABSOLUTE
    (NETDELAY A3/B (0.566))
  )
)
```

Delays

So far we have seen many different forms of delays. There are additional forms of delay specification. In general, delays can be specified as a set of one, two, three, six or twelve tokens that can be used to describe the following transition delays: `0->1`, `1->0`, `0->Z`, `Z->1`, `1->Z`, `Z->0`, `0->X`, `X->1`, `1->X`, `X->0`, `X->Z`, `Z->X`. The following table shows how fewer than twelve delay tokens are used to represent the twelve transitions.

Transition	2-values (v1 v2)	3-values (v1 v2 v3)	6-values (v1 v2 v3 v4 v5 v6)	12-values (v1 v2 v3 v4 v5 v6 v7 v8 v9 v10 v11 v12)
0->1	v1	v1	v1	v1
1->0	v2	v2	v2	v2
0->Z	v1	v3	v3	v3
Z->1	v1	v1	v4	v4
1->Z	v2	v3	v5	v5
Z->0	v2	v2	v6	v6
0->X	v1	min(v1,v3)	min(v1,v3)	v7
X->1	v1	v1	max(v1,v4)	v8
1->X	v2	min(v2,v3)	min(v2,v5)	v9
X->0	v2	v2	max(v2,v6)	v10
X->Z	max(v1,v2)	v3	max(v3,v5)	v11
Z->X	min(v1,v2)	min(v1,v2)	min(v6,v4)	v12

Table B-5 Mapping to twelve transition delays.

Here are some examples of these delays.

```
(DELAY
  (ABSOLUTE
```

```
    // 1-value delay:
    (IOPATH A Y (0.989))
    // 2-value delay:
    (IOPATH B Y (0.989) (0.891))
    // 6-value delay:
    (IOPATH CTRL Y (0.121) (0.119) (0.129)
                   (0.131) (0.112) (0.124))
    // 12-value delay:
    (COND RN == 1'b0
     (IOPATH C Y (0.330) (0.312) (0.330) (0.311) (0.328)
                 (0.321) (0.328) (0.320) (0.320)
                 (0.318) (0.318) (0.316)
     )
    )
    // In this 2-value delay, the first one is null
    // implying the annotator is not to change its value.
    (IOPATH RN Q () (0.129))
  )
)
```

Each delay token can, in turn, be written as one, two or three values as shown in the following examples.

```
(DELAY
 (ABSOLUTE
  // One value in a delay token:
  (IOPATH A Y (0.117))
  // The delay value, the pulse rejection limit
  // (r-limit) and X filter limit (e-limit) are same.

  // Two values in a delay (note no colon):
  (IOPATH (posedge CK) Q (0.12 0.15))
  // 0.12 is the delay value and 0.15 is the r-limit
  // and e-limit.
```

```
      // Three values in a delay:
      (IOPATH F1/Y AND1/A (0.339 0.1 0.15))
      // Path delay is 0.339, r-limit is 0.1 and
      // e-limit is 0.15.
    )
  )
```

Delay values in a single SDF file can be written using signed real numbers or as triplets of form:

```
(8.0 : 3.6 : 9.8)
```

to denote minimum, typical, maximum delays that represent the three process operating conditions of the design. The choice of which value is selected is made by the annotator typically based on a user-provided option. The values in the triplet form are optional, though it should have at least one. For example, the following are legal.

```
(::0.22)
(1.001: :0.998)
```

Values that are not specified are simply not annotated.

Timing Checks

Timing check limits are specified in the section that starts with the TIMINGCHECK keyword. In any of these checks, a COND construct can be used to specify conditional timing checks. In some cases, two additional conditional checks can be specified, SCOND and CCOND, that are associated with the *stamp event* and the *check event*.

Following are the set of checks:

i. SETUP: Setup timing check

ii. HOLD: Hold timing check

iii. SETUPHOLD: Setup and hold timing check

iv. RECOVERY: Recovery timing check

v. REMOVAL: Removal timing check

vi. RECREM: Recovery and removal timing check

vii. SKEW: Unidirectional skew timing check

viii. BIDIRECTSKEW: Bidirectional skew timing check

ix. WIDTH: Width timing check

x. PERIOD: Period timing check

xi. NOCHANGE: No-change timing check

Here are some examples.

```
(TIMINGCHECK
 // Setup check limit:
 (SETUP din (posedge clk) (2))

 // Hold check limit:
 (HOLD din (negedge clk) (0.445:0.445:0.445))

 // Conditional hold check limit :
 (HOLD (COND RST==1'b1 D) (posedge CLK) (1.15))
 // Hold check between D and positive edge of CLK, but
 // only when RST is 1.

 // Setup and hold check limit:
 (SETUPHOLD J CLK (1.2) (0.99))
 // 1.2 is the setup limit and 0.99 is the hold limit.

 // Conditional setup and hold limit:
 (SETUPHOLD D CLK (0.809) (0.591) (CCOND ~SE))
 // Condition applies with CLK for setup and
 // with D for hold.
```

```
    // Conditional setup and hold check limit:
    (SETUPHOLD (COND ~RST D) (posedge CLK) (1.452) (1.11))
    // Setup and hold check between D and positive edge
    // of CLK, but only when RST is low.

    // RECOVERY check limit:
    (RECOVERY SE (negedge CLK) (0.671))

    // Conditional removal check limit:
    (REMOVAL (COND ~LOAD CLEAR) CLK (2.001:2.1:2.145))
    // Removal check between CLEAR and CLK but only
    // when LOAD is low.

    // Recovery and removal check limit:
    (RECREM RST (negedge CLK) (1.1) (0.701))
    // 1.1 is the recovery limit and 0.701 is the
    // removal limit.

    // Skew conditional check limit:
    (SKEW (COND MMODE==1'b1 GNT) (posedge REQ) (3.2))

    // Bidirectional skew check limit:
    (BIDIRECTSKEW (posedge CLOCK1) (negedge TCK) (1.409))

    // Width check limit:
    (WIDTH (negedge RST) (12))

    // Period check limit:
    (PERIOD (posedge JTCLK) (13.33))

    // Nochange check limit:
    (NOCHANGE (posedge REQ) (negedge GNT) (2.5) (3.12))
  )
```

Labels

Labels are used to specify values for VHDL generics or Verilog HDL specify parameters.

```
(LABEL
  (ABSOLUTE
    (thold$d$clk (0.809))
    (tph$A$Y (0.553))
  )
)
```

Timing Environment

There are a number of constructs available that can be used to describe the timing environment of a design. However, these constructs are used for forward-annotation rather than backward-annotation, such as in logic synthesis tools. These are not described in this text.

B.2.1 Examples

We provide complete SDFs for two designs.

Full-adder

Here is the Verilog HDL netlist for a full-adder circuit.

```
module FA_STR (A, B, CIN, SUM, COUT);
  input A, B, CIN;
  output SUM, COUT;
  wire S1, S2, S3, S4, S5;

  XOR2X1 X1 (.Y(S1), .A(A), .B(B));
  XOR2X1 X2 (.Y(SUM), .A(S1), .B(CIN));
```

```
  AND2X1 A1 (.Y(S2), .A(A), .B(B));
  AND2X1 A2 (.Y(S3), .A(B), .B(CIN));
  AND2X1 A3 (.Y(S4), .A(A), .B(CIN));

  OR2X1 O1 (.Y(S5), .A(S2), .B(S3));
  OR2X1 O2 (.Y(COUT), .A(S4), .B(S5));
endmodule
```

Here is the complete corresponding SDF file produced by a timing analysis tool.

```
(DELAYFILE
  (SDFVERSION "OVI 2.1")
  (DESIGN "FA_STR")
  (DATE "Mon May 24 13:56:43 2004")
  (VENDOR "slow")
  (PROGRAM "CompanyName ToolName")
  (VERSION "V2.3")
  (DIVIDER /)
  // OPERATING CONDITION "slow"
  (VOLTAGE 1.35:1.35:1.35)
  (PROCESS "1.000:1.000:1.000")
  (TEMPERATURE 125.00:125.00:125.00)
  (TIMESCALE 1ns)
  (CELL
    (CELLTYPE "FA_STR")
    (INSTANCE)
    (DELAY
      (ABSOLUTE
        (INTERCONNECT A A3/A (0.000:0.000:0.000))
        (INTERCONNECT A A1/A (0.000:0.000:0.000))
        (INTERCONNECT A X1/A (0.000:0.000:0.000))
        (INTERCONNECT B A2/A (0.000:0.000:0.000))
        (INTERCONNECT B A1/B (0.000:0.000:0.000))
        (INTERCONNECT B X1/B (0.000:0.000:0.000))
        (INTERCONNECT CIN A3/B (0.000:0.000:0.000))
```

```
      (INTERCONNECT CIN A2/B (0.000:0.000:0.000))
      (INTERCONNECT CIN X2/B (0.000:0.000:0.000))
      (INTERCONNECT X2/Y SUM (0.000:0.000:0.000))
      (INTERCONNECT O2/Y COUT (0.000:0.000:0.000))
      (INTERCONNECT X1/Y X2/A (0.000:0.000:0.000))
      (INTERCONNECT A1/Y O1/A (0.000:0.000:0.000))
      (INTERCONNECT A2/Y O1/B (0.000:0.000:0.000))
      (INTERCONNECT A3/Y O2/A (0.000:0.000:0.000))
      (INTERCONNECT O1/Y O2/B (0.000:0.000:0.000))
    )
  )
)
(CELL
  (CELLTYPE "XOR2X1")
  (INSTANCE X1)
  (DELAY
    (ABSOLUTE
      (IOPATH A Y (0.197:0.197:0.197)
                  (0.190:0.190:0.190))
      (IOPATH B Y (0.209:0.209:0.209)
                  (0.227:0.227:0.227))
      (COND B==1'b1 (IOPATH A Y (0.197:0.197:0.197)
                                (0.190:0.190:0.190)))
      (COND A==1'b1 (IOPATH B Y (0.209:0.209:0.209)
                                (0.227:0.227:0.227)))
      (COND B==1'b0 (IOPATH A Y (0.134:0.134:0.134)
                                (0.137:0.137:0.137)))
      (COND A==1'b0 (IOPATH B Y (0.150:0.150:0.150)
                                (0.163:0.163:0.163)))
    )
  )
)
(CELL
  (CELLTYPE "XOR2X1")
  (INSTANCE X2)
  (DELAY
    (ABSOLUTE
```

```
      (IOPATH (posedge A) Y (0.204:0.204:0.204)
                            (0.196:0.196:0.196))
      (IOPATH (negedge A) Y (0.198:0.198:0.198)
                            (0.190:0.190:0.190))
      (IOPATH B Y (0.181:0.181:0.181)
                  (0.201:0.201:0.201))
      (COND B==1'b1 (IOPATH A Y (0.198:0.198:0.198)
                                (0.196:0.196:0.196)))
      (COND A==1'b1 (IOPATH B Y (0.181:0.181:0.181)
                                (0.201:0.201:0.201)))
      (COND B==1'b0 (IOPATH A Y (0.135:0.135:0.135)
                                (0.140:0.140:0.140)))
      (COND A==1'b0 (IOPATH B Y (0.122:0.122:0.122)
                                (0.139:0.139:0.139)))
    )
   )
  )

  (CELL
   (CELLTYPE "AND2X1")
   (INSTANCE A1)
   (DELAY
    (ABSOLUTE
     (IOPATH A Y (0.147:0.147:0.147)
                 (0.157:0.157:0.157))
     (IOPATH B Y (0.159:0.159:0.159)
                 (0.173:0.173:0.173))
    )
   )
  )
  (CELL
   (CELLTYPE "AND2X1")
   (INSTANCE A2)
   (DELAY
    (ABSOLUTE
     (IOPATH A Y (0.148:0.148:0.148)
                 (0.157:0.157:0.157))
```

```
      (IOPATH B Y (0.160:0.160:0.160)
                  (0.174:0.174:0.174))
    )
   )
  )
  (CELL
   (CELLTYPE "AND2X1")
   (INSTANCE A3)
   (DELAY
    (ABSOLUTE
     (IOPATH A Y (0.147:0.147:0.147)
                 (0.157:0.157:0.157))
     (IOPATH B Y (0.159:0.159:0.159)
                 (0.173:0.173:0.173))
    )
   )
  )
  (CELL
   (CELLTYPE "OR2X1")
   (INSTANCE O1)
   (DELAY
    (ABSOLUTE
     (IOPATH A Y (0.138:0.138:0.138)
                 (0.203:0.203:0.203))
     (IOPATH B Y (0.151:0.151:0.151)
                 (0.223:0.223:0.223))
    )
   )
  )
  (CELL
   (CELLTYPE "OR2X1")
   (INSTANCE O2)
   (DELAY
    (ABSOLUTE
     (IOPATH A Y (0.126:0.126:0.126)
                 (0.191:0.191:0.191))
     (IOPATH B Y (0.136:0.136:0.136)
```

```
                    (0.212:0.212:0.212))
      )
    )
  )
)
```

All delays in the INTERCONNECTs are 0 as this is pre-layout data and ideal interconnects are modeled.

Decade Counter

Here is the Verilog HDL model for a decade counter.

```
module DECADE_CTR (COUNT, Z);
  input COUNT;
  output [0:3] Z;
  wire S1, S2;

  AND2X1 a1 (.Y(S1), .A(Z[2]), .B(Z[1]));

  JKFFX1
    JK1 (.J(1'b1), .K(1'b1), .CK(COUNT),
         .Q(Z[0]), .QN()),
    JK2 (.J(S2), .K(1'b1), .CK(Z[0]), .Q(Z[1]), .QN()),
    JK3 (.J(1'b1), .K(1'b1), .CK(Z[1]),
         .Q(Z[2]), .QN()),
    JK4 (.J(S1), .K(1'b1), .CK(Z[0]),
         .Q(Z[3]), .QN(S2));
endmodule
```

The complete corresponding SDF follows.

```
(DELAYFILE
  (SDFVERSION "OVI 2.1")
  (DESIGN "DECADE_CTR")
  (DATE "Mon May 24 14:30:17 2004")
```

```
(VENDOR "Star Galaxy Automation, Inc.")
(PROGRAM "MyCompanyName ToolTime")
(VERSION "V2.3")
(DIVIDER /)
// OPERATING CONDITION "slow"
(VOLTAGE 1.35:1.35:1.35)
(PROCESS "1.000:1.000:1.000")
(TEMPERATURE 125.00:125.00:125.00)
(TIMESCALE 1ns)
(CELL
  (CELLTYPE "DECADE_CTR")
  (INSTANCE)
  (DELAY
    (ABSOLUTE
      (INTERCONNECT COUNT JK1/CK (0.191:0.191:0.191))
      (INTERCONNECT JK1/Q Z\[0\] (0.252:0.252:0.252))
      (INTERCONNECT JK2/Q Z\[1\] (0.186:0.186:0.186))
      (INTERCONNECT JK3/Q Z\[2\] (0.18:0.18:0.18))
      (INTERCONNECT JK4/Q Z\[3\] (0.195:0.195:0.195))
      (INTERCONNECT JK3/Q a1/A (0.175:0.175:0.175))
      (INTERCONNECT JK2/Q a1/B (0.207:0.207:0.207))
      (INTERCONNECT JK4/QN JK2/J (0.22:0.22:0.22))
      (INTERCONNECT JK1/Q JK2/CK (0.181:0.181:0.181))
      (INTERCONNECT JK2/Q JK3/CK (0.193:0.193:0.193))
      (INTERCONNECT a1/Y JK4/J (0.224:0.224:0.224))
      (INTERCONNECT JK1/Q JK4/CK (0.218:0.218:0.218))
    )
  )
)
(CELL
  (CELLTYPE "AND2X1")
  (INSTANCE a1)
  (DELAY
    (ABSOLUTE
      (IOPATH A Y (0.179:0.179:0.179)
                  (0.186:0.186:0.186))
      (IOPATH B Y (0.190:0.190:0.190)
```

```
                    (0.210:0.210:0.210))
      )
    )
  )
  (CELL
    (CELLTYPE "JKFFX1")
    (INSTANCE JK1)
    (DELAY
      (ABSOLUTE
        (IOPATH (posedge CK) Q (0.369:0.369:0.369)
                               (0.470:0.470:0.470))
        (IOPATH (posedge CK) QN (0.280:0.280:0.280)
                                (0.178:0.178:0.178))
      )
    )
    (TIMINGCHECK
      (SETUP (posedge J) (posedge CK)
        (0.362:0.362:0.362))
      (SETUP (negedge J) (posedge CK)
        (0.220:0.220:0.220))
      (HOLD (posedge J) (posedge CK)
        (-0.272:-0.272:-0.272))
      (HOLD (negedge J) (posedge CK)
        (-0.200:-0.200:-0.200))
      (SETUP (posedge K) (posedge CK)
        (0.170:0.170:0.170))
      (SETUP (negedge K) (posedge CK)
        (0.478:0.478:0.478))
      (HOLD (posedge K) (posedge CK)
        (-0.158:-0.158:-0.158))
      (HOLD (negedge K) (posedge CK)
        (-0.417:-0.417:-0.417))
      (WIDTH (negedge CK)
        (0.337:0.337:0.337))
      (WIDTH (posedge CK) (0.148:0.148:0.148))
    )
  )
```

```
(CELL
  (CELLTYPE "JKFFX1")
  (INSTANCE JK2)
  (DELAY
    (ABSOLUTE
      (IOPATH (posedge CK) Q (0.409:0.409:0.409)
                          (0.512:0.512:0.512))
      (IOPATH (posedge CK) QN (0.326:0.326:0.326)
                           (0.222:0.222:0.222))
    )
  )
  (TIMINGCHECK
    (SETUP (posedge J) (posedge CK)
      (0.348:0.348:0.348))
    (SETUP (negedge J) (posedge CK)
      (0.227:0.227:0.227))
    (HOLD (posedge J) (posedge CK)
      (-0.257:-0.257:-0.257))
    (HOLD (negedge J) (posedge CK)
      (-0.209:-0.209:-0.209))
    (SETUP (posedge K) (posedge CK)
      (0.163:0.163:0.163))
    (SETUP (negedge K) (posedge CK)
      (0.448:0.448:0.448))
    (HOLD (posedge K) (posedge CK)
      (-0.151:-0.151:-0.151))
    (HOLD (negedge K) (posedge CK)
      (-0.392:-0.392:-0.392))
    (WIDTH (negedge CK) (0.337:0.337:0.337))
    (WIDTH (posedge CK) (0.148:0.148:0.148))
  )
)
(CELL
  (CELLTYPE "JKFFX1")
  (INSTANCE JK3)
  (DELAY
    (ABSOLUTE
```

```
      (IOPATH (posedge CK) Q (0.378:0.378:0.378)
                             (0.485:0.485:0.485))
      (IOPATH (posedge CK) QN (0.324:0.324:0.324)
                              (0.221:0.221:0.221))
    )
   )
   (TIMINGCHECK
    (SETUP (posedge J) (posedge CK)
     (0.339:0.339:0.339))
    (SETUP (negedge J) (posedge CK)
     (0.211:0.211:0.211))
    (HOLD (posedge J) (posedge CK)
     (-0.249:-0.249:-0.249))
    (HOLD (negedge J) (posedge CK)
     (-0.192:-0.192:-0.192))
    (SETUP (posedge K) (posedge CK)
     (0.163:0.163:0.163))
    (SETUP (negedge K) (posedge CK)
     (0.449:0.449:0.449))
    (HOLD (posedge K) (posedge CK)
     (-0.152:-0.152:-0.152))
    (HOLD (negedge K) (posedge CK)
     (-0.393:-0.393:-0.393))
    (WIDTH (negedge CK) (0.337:0.337:0.337))
    (WIDTH (posedge CK) (0.148:0.148:0.148))
   )
  )
  (CELL
   (CELLTYPE "JKFFX1")
   (INSTANCE JK4)
   (DELAY
    (ABSOLUTE
     (IOPATH (posedge CK) Q (0.354:0.354:0.354)
                            (0.464:0.464:0.464))
     (IOPATH (posedge CK) QN (0.364:0.364:0.364)
                             (0.256:0.256:0.256))
    )
```

```
    )
    (TIMINGCHECK
      (SETUP (posedge J) (posedge CK)
        (0.347:0.347:0.347))
      (SETUP (negedge J) (posedge CK)
        (0.226:0.226:0.226))
      (HOLD (posedge J) (posedge CK)
        (-0.256:-0.256:-0.256))
      (HOLD (negedge J) (posedge CK)
        (-0.208:-0.208:-0.208))
      (SETUP (posedge K) (posedge CK)
        (0.163:0.163:0.163))
      (SETUP (negedge K) (posedge CK)
        (0.448:0.448:0.448))
      (HOLD (posedge K) (posedge CK)
        (-0.151:-0.151:-0.151))
      (HOLD (negedge K) (posedge CK)
        (-0.392:-0.392:-0.392))
      (WIDTH (negedge CK) (0.337:0.337:0.337))
      (WIDTH (posedge CK) (0.148:0.148:0.148))
    )
  )
)
```

B.3 The Annotation Process

In this section, we describe how the annotation of the SDF occurs to an HDL description. SDF annotation can be performed by a number of tools, such as logic synthesis, simulation and static timing analysis; the SDF annotator is the component of these tools that reads the SDF, interprets and annotates the timing values to the design. It is assumed that the SDF file is created using information that is consistent with the HDL model and that the same HDL model is used during backannotation. Additionally, it is the responsibility of the SDF annotator to ensure that the timing values in the SDF are interpreted correctly.

The SDF annotator annotates the backannotation timing generics and parameters. It reports any errors if there is any noncompliance to the standard, either in syntax or in the mapping process. If certain SDF constructs are not supported by an SDF annotator, no errors are produced - the annotator simply ignores these.

If the SDF annotator fails to modify a backannotation timing generic, then the value of the generic is not modified during the backannotation process, that is, it is left unchanged.

In a simulation tool, backannotation typically occurs just following the elaboration phase and directly preceding negative constraint delay calculation.

B.3.1 Verilog HDL

In Verilog HDL, the primary mechanism for annotation is the specify block. A specify block can specify path delays and timing checks. Actual delay values and timing check limit values are specified via the SDF file. The mapping is an industry standard and is defined in IEEE Std 1364.

Specify path delays, specparam values, timing check constraint limits and interconnect delays are among the information obtained from an SDF file and annotated in a specify block of a Verilog HDL module. Other constructs in an SDF file are ignored when annotating to a Verilog HDL model. The LABEL section in SDF defines specparam values. Backannotation is done by matching SDF constructs to corresponding Verilog HDL declarations and then replacing the existing timing values with those in the SDF file.

Here is a table that shows how SDF delay values are mapped to Verilog HDL delay values.

Verilog transition	1-value (v1)	2-values (v1 v2)	3-values (v1 v2 v3)	6-values (v1 v2 v3 v4 v5 v6)	12-values (v1 v2 v3 v4 v5 v6 v7 v8 v9 v10 v11 v12)
0->1	v1	v1	v1	v1	v1
1->0	v1	v2	v2	v2	v2
0->z	v1	v1	v3	v3	v3
z->1	v1	v1	v1	v4	v4
1->z	v1	v2	v3	v5	v5
z->0	v1	v2	v2	v6	v6
0->x	v1	v1	min(v1 v3)	min(v1 v3)	v7
x->1	v1	v1	v1	max(v1 v4)	v8
1->x	v1	v2	min (v2 v3)	min(v2 v5)	v9
x->0	v1	v2	v2	max(v2 v6)	v10
x->z	v1	max(v1 v2)	v3	max(v3 v5)	v11
z->x	v1	min(v1 v2)	min(v1 v2)	min(v4 v6)	v12

Table B-6 Mapping SDF delays to Verilog HDL delays.

The following table describes the mapping of SDF constructs to Verilog HDL constructs.

Kinds	SDF construct	Verilog HDL
Propagation delay	IOPATH	Specify paths
Input setup time	SETUP	**$setup**, **$setuphold**
Input hold time	HOLD	**$hold**, **$setuphold**
Input setup and hold	SETUPHOLD	**$setup**, **$hold**, **$setuphold**
Input recovery time	RECOVERY	**$recovery**
Input removal time	REMOVAL	**$removal**
Recovery and removal	RECREM	**$recovery**, **$removal**, **$recrem**
Period	PERIOD	**$period**
Pulse width	WIDTH	**$width**
Input skew time	SKEW	**$skew**
No-change time	NOCHANGE	**$nochange**
Port delay	PORT	Interconnect delay
Net delay	NETDELAY	Interconnect delay
Interconnect delay	INTERCONNECT	Interconnect delay
Device delay	DEVICE_DELAY	Specify paths
Path pulse limit	PATHPULSE	Specify path pulse limit
Path pulse limit	PATHPULSEPERCENT	Specify path pulse limit

Table B-7 Mapping of SDF to Verilog HDL.

See later section for examples.

B.3.2 VHDL

Annotation of SDF to VHDL is an industry standard. It is defined in the IEEE standard for VITAL ASIC Modeling Specification, IEEE Std 1076.4; one of the components of this standard describes the annotation of SDF delays into ASIC libraries. Here, we present only the relevant part of the VITAL standard as it relates to SDF mapping.

SDF is used to modify backannotation timing generics in a VITAL-compliant model directly. Timing data can be specified only for a VITAL-compliant model using SDF. There are two ways to pass timing data into a VHDL model: via configurations, or directly into simulation. The SDF annotation process consists of mapping SDF constructs and corresponding generics in a VITAL-compliant model during simulation.

In a VITAL-compliant model, there are rules on how generics are to be named and declared that ensures that a mapping can be established between the timing generics of a model and the corresponding SDF timing information.

A timing generic is made up of a generic name and its type. The name specifies the kind of timing information and the type of the generic specifies the kind of timing value. If the name of generic does not follow the VITAL standard, then it is not a timing generic and does not get annotated.

Here is the table showing how SDF delay values are mapped to VHDL delays.

VHDL transition	1-value (v1)	2-values (v1 v2)	3-values (v1 v2 v3)	6-values (v1 v2 v3 v4 v5 v6)	12-values (v1 v2 v3 v4 v5 v6 v7 v8 v9 v10 v11 v12)
0->1	v1	v1	v1	v1	v1
1->0	v1	v2	v2	v2	v2
0->z	v1	v1	v3	v3	v3
z->1	v1	v1	v1	v4	v4
1->z	v1	v2	v3	v5	v5
z->0	v1	v2	v2	v6	v6
0->x	-	-	-	-	v7
x->1	-	-	-	-	v8
1->x	-	-	-	-	v9
x->0	-	-	-	-	v10
x->z	-	-	-	-	v11
z->x	-	-	-	-	v12

Table B-8 Mapping SDF delays to VHDL delays.

In VHDL, timing information is backannotated via generics. Generic names follow a certain convention so as to be consistent or derived from SDF constructs. With each of the timing generic names, an optional suffix of a conditioned edge can be specified. The edge specifies an edge associated with the timing information.

Table B-9 shows the different kinds of timing generic names.

Kinds	SDF construct	VHDL generic
Propagation delay	IOPATH	**tpd**_*InputPort_OutputPort* [_*condition*]
Input setup time	SETUP	**tsetup**_*TestPort_RefPort* [_*condition*]
Input hold time	HOLD	**thold**_*TestPort_RefPort* [_*condition*]
Input recovery time	RECOVERY	**trecovery**_*TestPort_RefPort* [_*condition*]
Input removal time	REMOVAL	**tremoval**_*TestPort_RefPort* [_*condition*]
Period	PERIOD	**tperiod**_*InputPort* [_*condition*]
Pulse width	WIDTH	**tpw**_*InputPort* [_*condition*]
Input skew time	SKEW	**tskew**_*FirstPort_SecondPort* [_*condition*]
No-change time	NOCHANGE	**tncsetup**_*TestPort_RefPort* [_*condition*] **tnchold**_*TestPort_RefPort* [_*condition*]
Interconnect path delay	PORT	**tipd**_*InputPort*
Device delay	DEVICE	**tdevice**_*InstanceName* [_*OutputPort*]
Internal signal delay		**tisd**_*InputPort_ClockPort*
Biased propagation delay		**tbpd**_*InputPort_OutputPort_ClockPort* [_*condition*]
Internal clock delay		**ticd**_*ClockPort*

Table B-9 Mapping of SDF to VHDL generics.

B.4 Mapping Examples

Here are examples of mapping SDF constructs to VHDL generics and Verilog HDL declarations.

Propagation Delay

- Propagation delay from input port A to output port Y with a rise time of 0.406 and a fall of 0.339.

```
// SDF:
(IOPATH A Y (0.406) (0.339))

-- VHDL generic:
tpd_A_Y : VitalDelayType01;

// Verilog HDL specify path:
(A *> Y) = (tplh$A$Y, tphl$A$Y);
```

- Propagation delay from input port OE to output port Y with a rise time of 0.441 and a fall of 0.409. The minimum, nominal and maximum delays are identical.

```
// SDF:
(IOPATH OE Y (0.441:0.441:0.441) (0.409:0.409:0.409))

-- VHDL generic:
tpd_OE_Y : VitalDelayType01Z;

// Verilog HDL specify path:
(OE *> Y) = (tplh$OE$Y, tphl$OE$Y);
```

- Conditional propagation delay from input port S0 to output port Y.

```
// SDF:
(COND A==0 && B==1 && S1==0
  (IOPATH S0 Y (0.062:0.062:0.062) (0.048:0.048:0.048)
  )
)
```

```
-- VHDL generic:
tpd_S0_Y_A_EQ_0_AN_B_EQ_1_AN_S1_EQ_0 :
  VitalDelayType01;

// Verilog HDL specify path:
if ((A == 1'b0) && (B == 1'b1) && (S1 == 1'b0))
  (S0 *> Y) = (tplh$S0$Y, tphl$S0$Y);
```

- Conditional propagation delay from input port A to output port Y.

```
// SDF:
(COND B == 0
  (IOPATH A Y (0.130) (0.098)
  )
)

-- VHDL generic:
tpd_A_Y_B_EQ_0 : VitalDelayType01;

// Verilog HDL specify path:
if (B == 1'b0)
  (A *> Y) = 0;
```

- Propagation delay from input port CK to output port Q.

```
// SDF:
(IOPATH CK Q (0.100:0.100:0.100) (0.118:0.118:0.118))

-- VHDL generic:
tpd_CK_Q : VitalDelayType01;

// Verilog HDL specify path:
(CK *> Q) = (tplh$CK$Q, tphl$CK$Q);
```

- Conditional propagation delay from input port A to output port Y.

```
// SDF:
(COND B == 1
  (IOPATH A Y (0.062:0.062:0.062) (0.048:0.048:0.048)
  )
)

-- VHDL generic:
tpd_A_Y_B_EQ_1 : VitalDelayType01;

// Verilog HDL specify path:
if (B == 1'b1)
  (A *> Y) = (tplh$A$Y, tphl$A$Y);
```

- Propagation delay from input port CK to output port ECK.

```
// SDF:
(IOPATH CK ECK (0.097:0.097:0.097))

-- VHDL generic:
tpd_CK_ECK : VitalDelayType01;

// Verilog HDL specify path:
(CK *> ECK) = (tplh$CK$ECK, tphl$CK$ECK);
```

- Conditional propagation delay from input port CI to output port S.

```
// SDF:
(COND (A == 0 && B == 0) || (A == 1 && B == 1)
  (IOPATH CI S (0.511) (0.389)
```

```
  )
)

-- VHDL generic:
tpd_CI_S_OP_A_EQ_0_AN_B_EQ_0_CP_OR_OP_A_EQ_1_AN_B_EQ_1_CP:
  VitalDelayType01;

// Verilog HDL specify path:
if ((A == 1'b0 && B == 1'b0) || (A == 1'b1 && B == 1'b1))
  (CI *> S)  = (tplh$CI$S, tphl$CI$S);
```

- Conditional propagation delay from input port CS to output port S.

```
// SDF:
(COND (A == 1 ^ B == 1 ^ CI1 == 1) &&
     !(A == 1 ^ B == 1 ^ CI0 == 1)
  (IOPATH CS S (0.110) (0.120) (0.120)
               (0.110) (0.119) (0.120)
  )
)

-- VHDL generic:
tpd_CS_S_OP_A_EQ_1_XOB_B_EQ_1_XOB_CI1_EQ_1_CP_AN_NT_
OP_A_EQ_1_XOB_B_EQ_1_XOB_CI0_EQ_1_CP:
  VitalDelayType01;

// Verilog HDL specify path:
if ((A == 1'b1 ^ B == 1'b1 ^ CI1N == 1'b0) &&
    !(A == 1'b1 ^ B == 1'b1 ^ CI0N == 1'b0))
  (CS  *> S)  = (tplh$CS$S, tphl$CS$S);
```

- Conditional propagation delay from input port A to output port ICO.

```
// SDF:
(COND B == 1 (IOPATH A ICO (0.690)))

-- VHDL generic:
tpd_A_ICO_B_EQ_1 : VitalDelayType01;

// Verilog HDL specify path:
if (B == 1'b1)
  (A  *> ICO) = (tplh$A$ICO,  tphl$A$ICO);
```

- Conditional propagation delay from input port A to output port CO.

```
// SDF:
(COND (B == 1 ^ C == 1) && (D == 1 ^ ICI == 1)
  (IOPATH A CO (0.263)
  )
)

-- VHDL generic:
tpd_A_CO_OP_B_EQ_1_XOB_C_EQ_1_CP_AN_OP_D_EQ_1_XOB_ICI_E
Q_1_CP: VitalDelayType01;

// Verilog HDL specify path:
if ((B == 1'b1 ^ C == 1'b1) && (D == 1'b1 ^ ICI == 1'b1))
  (A *> CO) = (tplh$A$CO,  tphl$A$CO);
```

- Delay from positive edge of CK to Q.

```
// SDF:
(IOPATH (posedge CK) Q (0.410:0.410:0.410)
  (0.290:0.290:0.290))
```

```
-- VHDL generic:
tpd_CK_Q_posedge_noedge : VitalDelayType01;

// Verilog HDL specify path:
(posedge CK *> Q) = (tplh$CK$Q, tphl$CK$Q);
```

Input Setup Time

- Setup time between posedge of D and posedge of CK.

```
// SDF:
(SETUP (posedge D) (posedge CK) (0.157:0.157:0.157))

-- VHDL generic:
tsetup_D_CK_posedge_posedge: VitalDelayType;

// Verilog HDL timing check task:
$setup(posedge CK, posedge D, tsetup$D$CK, notifier);
```

- Setup between negedge of D and posedge of CK.

```
// SDF:
(SETUP (negedge D) (posedge CK) (0.240))

-- VHDL generic:
tsetup_D_CK_negedge_posedge: VitalDelayType;

// Verilog HDL timing check task:
$setup(posedge CK, negedge D, tsetup$D$CK, notifier);
```

- Setup time between posedge of input E with posedge of reference CK.

```
// SDF:
(SETUP (posedge E) (posedge CK) (-0.043:-0.043:-0.043))

-- VHDL generic:
tsetup_E_CK_posedge_posedge : VitalDelayType;

// Verilog HDL timing check task:
$setup(posedge CK, posedge E, tsetup$E$CK, notifier);
```

- Setup time between negedge of input E and posedge of reference CK.

```
// SDF:
(SETUP (negedge E) (posedge CK) (0.101) (0.098))

-- VHDL generic:
tsetup_E_CK_negedge_posedge : VitalDelayType;

// Verilog HDL timing check task:
$setup(posedge CK, negedge E, tsetup$E$CK, notifier);
```

- Conditional setup time between SE and CK.

```
// SDF:
(SETUP (cond E != 1 SE) (posedge CK) (0.155) (0.135))

-- VHDL generic:
tsetup_SE_CK_E_NE_1_noedge_posedge : VitalDelayType;

// Verilog HDL timing check task:
$setup(posedge CK &&& (E != 1'b1), SE, tsetup$SE$CK,
      notifier);
```

Input Hold Time

- Hold time between posedge of D and posedge of CK.

```
// SDF:
(HOLD (posedge D) (posedge CK) (-0.166:-0.166:-0.166))

-- VHDL generic:
thold_D_CK_posedge_posedge: VitalDelayType;

// Verilog HDL timing check task:
$hold (posedge CK, posedge D, thold$D$CK, notifier);
```

- Hold time between RN and SN.

```
// SDF:
(HOLD (posedge RN) (posedge SN) (-0.261:-0.261:-0.261))

-- VHDL generic:
thold_RN_SN_posedge_posedge: VitalDelayType;

// Verilog HDL timing check task:
$hold (posedge SN, posedge RN, thold$RN$SN, notifier);
```

- Hold time between input port SI and reference port CK.

```
// SDF:
(HOLD (negedge SI) (posedge CK) (-0.110:-0.110:-0.110))

-- VHDL generic:
thold_SI_CK_negedge_posedge: VitalDelayType;

// Verilog HDL timing check task:
$hold (posedge CK, negedge SI, thold$SI$CK, notifier);
```

- Conditional hold time between E and posedge of CK.

```
// SDF:
(HOLD (COND SE ^ RN == 0 E) (posedge CK))

-- VHDL generic:
thold_E_CK_SE_XOB_RN_EQ_0_noedge_posedge:
 VitalDelayType;

// Verilog HDL timing check task:
$hold (posedge CK &&& (SE ^ RN == 0), posedge E,
      thold$E$CK, NOTIFIER);
```

Input Setup and Hold Time

- Setup and hold timing check between D and CLK. It is a conditional check. The first delay value is the setup time and the second delay value is the hold time.

```
// SDF:
(SETUPHOLD (COND SE ^ RN == 0 D) (posedge CLK)
 (0.69) (0.32))

-- VHDL generic (split up into separate setup and hold):
tsetup_D_CK_SE_XOB_RN_EQ_0_noedge_posedge:
 VitalDelayType;
thold_D_CK_SE_XOB_RN_EQ_0_noedge_posedge:
 VitalDelayType;

-- Verilog HDL timing check (it can either be split up or
-- kept as one construct depending on what appears in the
-- Verilog HDL model):
$setuphold(posedge CK &&& (SE ^ RN == 1'b0)), posedge D,
          tsetup$D$CK, thold$D$CK, notifier);
-- Or as:
```

```
$setup(posedge CK &&& (SE ^ RN == 1'b0)), posedge D,
      tsetup$D$CK, notifier);
$hold(posedge CK &&& (SE ^ RN == 1'b0)), posedge D,
     thold$D$CK, notifier);
```

Input Recovery Time

- Recovery time between CLKA and CLKB.

```
// SDF:
(RECOVERY (posedge CLKA) (posedge CLKB)
  (1.119:1.119:1.119))

-- VHDL generic:
trecovery_CLKA_CLKB_posedge_posedge: VitalDelayType;

// Verilog timing check task:
$recovery (posedge CLKB, posedge CLKA,
          trecovery$CLKB$CLKA, notifier);
```

- Conditional recovery time between posedge of CLKA and posedge of CLKB.

```
// SDF:
(RECOVERY (posedge CLKB)
  (COND ENCLKBCLKArec  (posedge CLKA)) (0.55:0.55:0.55)
)

-- VHDL generic:
trecovery_CLKB_CLKA_ENCLKBCLKArec_EQ_1_posedge_
posedge: VitalDelayType;

// Verilog timing check task:
$recovery (posedge CLKA && ENCLKBCLKArec, posedge CLKB,
           trecovery$CLKA$CLKB, notifier);
```

- Recovery time between SE and CK.

```
// SDF:
(RECOVERY SE (posedge CK) (1.901))

-- VHDL generic:
trecovery_SE_CK_noedge_posedge: VitalDelayType;

// Verilog timing check task:
$recovery (posedge CK, SE, trecovery$SE$CK, notifier);
```

- Recovery time between RN and CK.

```
// SDF:
(RECOVERY (COND D == 0 (posedge RN)) (posedge CK) (0.8))

-- VHDL generic:
trecovery_RN_CK_D_EQ_0_posedge_posedge:
 VitalDelayType;

// Verilog timing check task:
$recovery (posedge CK && (D == 0), posedge RN,
          trecovery$RN$CK, notifier);
```

Input Removal Time

- Removal time between posedge of E and negedge of CK.

```
// SDF:
(REMOVAL (posedge E) (negedge CK) (0.4:0.4:0.4))

-- VHDL generic:
tremoval_E_CK_posedge_negedge: VitalDelayType;
```

```
// Verilog timing check task:
$removal (negedge CK, posedge E, tremoval$E$CK,
         notifier);
```

- Conditional removal time between posedge of CK and SN.

```
// SDF:
(REMOVAL (COND D != 1'b1 SN) (posedge CK) (1.512))

-- VHDL generic:
tremoval_SN_CK_D_NE_1_noedge_posedge : VitalDelayType;

// Verilog timing check task:
$removal (posedge CK &&& (D != 1'b1), SN,
         tremoval$SN$CK, notifier);
```

Period

- Period of input CLKB.

```
// SDF:
(PERIOD CLKB (0.803:0.803:0.803))

-- VHDL generic:
tperiod_CLKB: VitalDelayType;

// Verilog timing check task:
$period (CLKB, tperiod$CLKB);
```

- Period of input port EN.

```
// SDF:
(PERIOD EN (1.002:1.002:1.002))
```

```
-- VHDL generic:
tperiod_EN : VitalDelayType;

// Verilog timing check task:
$period (EN, tperiod$EN);
```

- Period of input port TCK.

```
// SDF:
(PERIOD (posedge TCK) (0.220))

-- VHDL generic:
tperiod_TCK_posedge: VitalDelayType;

// Verilog timing check task:
$period (posedge TCK, tperiod$TCK);
```

Pulse Width

- Pulse width of high pulse of CK.

```
// SDF:
(WIDTH (posedge CK) (0.103:0.103:0.103))

-- VHDL generic:
tpw_CK_posedge: VitalDelayType;

// Verilog timing check task:
$width (posedge CK, tminpwh$CK, 0, notifier);
```

- Pulse width for a low pulse CK.

```
// SDF:
(WIDTH (negedge CK) (0.113:0.113:0.113))
```

```
-- VHDL generic:
tpw_CK_negedge: VitalDelayType;

// Verilog timing check task:
$width (negedge CK, tminpwl$CK, 0, notifier);
```

- Pulse width for a high pulse on RN.

```
// SDF:
(WIDTH (posedge RN) (0.122))

-- VHDL generic:
tpw_RN_posedge: VitalDelayType;

// Verilog timing check task:
$width (posedge RN, tminpwh$RN, 0, notifier);
```

Input Skew Time

- Skew between CK and TCK.

```
// SDF:
(SKEW (negedge CK) (posedge TCK) (0.121))

-- VHDL generic:
tskew_CK_TCK_negedge_posedge: VitalDelayType;

// Verilog timing check task:
$skew (posedge TCK, negedge CK, tskew$TCK$CK, notifier);
```

- Skew between SE and negedge of CK.

```
// SDF:
(SKEW SE (negedge CK) (0.386:0.386:0.386))
```

```
-- VHDL generic:
tskew_SE_CK_noedge_negedge: VitalDelayType;

// Verilog HDL timing check task:
$skew (negedge CK, SE, tskew$SE$CK, notifier);
```

No-change Setup Time

The SDF NOCHANGE construct maps to both *tncsetup* and *tnchold* VHDL generics.

- No-change setup time between D and negedge CK.

```
// SDF:
(NOCHANGE D (negedge CK) (0.343:0.343:0.343))

-- VHDL generic:
tncsetup_D_CK_noedge_negedge: VitalDelayType;
tnchold_D_CK_noedge_negedge: VitalDelayType;

// Verilog HDL timing check task:
$nochange (negedge CK, D, tnochange$D$CK, notifier);
```

No-change Hold Time

The SDF NOCHANGE construct maps to both *tncsetup* and *tnchold* VHDL generics.

- Conditional no-change hold time between E and CLKA.

```
// SDF:
(NOCHANGE (COND RST == 1'b1 (posedge E)) (posedge CLKA)
  (0.312))
```

```
-- VHDL generic:
tnchold_E_CLKA_RST_EQ_1_posedge_posedge:
 VitalDelayType;
tncsetup_E_CLKA_RST_EQ_1_posedge_posedge:
 VitalDelayType;

// Verilog HDL timing check task:
$nochange (posedge CLKA &&& (RST == 1'b1), posedge E,
         tnochange$E$CLKA, notifier);
```

Port Delay

- Delay to port OE.

```
// SDF:
(PORT OE (0.266))

-- VHDL generic:
tipd_OE: VitalDelayType01;

// Verilog HDL:
No explicit Verilog declaration.
```

- Delay to port RN.

```
// SDF:
(PORT RN (0.201:0.205:0.209))

-- VHDL generic:
tipd_RN : VitalDelayType01;

// Verilog HDL:
No explicit Verilog declaration.
```

Net Delay

- Delay on net connected to port CKA.

```
// SDF:
(NETDELAY CKA (0.134))

-- VHDL generic:
tipd_CKA: VitalDelayType01;

// Verilog HDL:
No explicit Verilog declaration.
```

Interconnect Path Delay

- Interconnect path delay from port Y to port D.

```
// SDF:
(INTERCONNECT bcm/credit_manager/U304/Y
 bcm/credit_manager/frame_in/PORT0_DOUT_Q_reg_26_/D
 (0.002:0.002:0.002) (0.002:0.002:0.002))

-- VHDL generic of instance
-- bcm/credit_manager/frame_in/PORT0_DOUT_Q_reg_26_:
tipd_D: VitalDelayType01;
-- The "from" port does not contribute to the timing
-- generic name.

// Verilog HDL:
No explicit Verilog declaration.
```

Device Delay

- Device delay of output SM of instance uP.

```
// SDF:
(INSTANCE uP) . . . (DEVICE SM . . .

-- VHDL generic:
tdevice_uP_SM

// Verilog specify paths:
// All specify paths to output SM.
```

B.5 Complete Syntax

Here is the complete syntax[1] for SDF shown using the BNF form. Terminal names are in uppercase, keywords are in bold uppercase but are case insensitive. The start terminal is `delay_file`.

```
absolute_deltype ::= ( ABSOLUTE del_def { del_def } )

alphanumeric ::=
    a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p |
      q | r | s | t | u | v | w | x | y | z
  | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P |
      Q | R | S | T | U | V | W | X | Y | Z
  | _ | $
  | decimal_digit

any_character ::=
    character
  | special_character
  | \"
```

1. The syntax is reprinted with permission from IEEE Std. 1497-2001, Copyright 2001, by IEEE. All rights reserved.

```
arrival_env ::=
  ( ARRIVAL [ port_edge ] port_instance rvalue rvalue
    rvalue rvalue )

bidirectskew_timing_check ::=
  ( BIDIRECTSKEW port_tchk port_tchk value value )

binary_operator ::=
    +
  | -
  | *
  | /
  | %
  | ==
  | !=
  | ===
  | !==
  | &&
  | ||
  | <
  | <=
  | >
  | >=
  | &
  | |
  | ^
  | ^~
  | ~^
  | >>
  | <<

bus_net ::= hierarchical_identifier [ integer : integer ]

bus_port ::= hierarchical_identifier [ integer : integer ]

ccond ::= ( CCOND [ qstring ] timing_check_condition )

cell ::= ( CELL celltype cell_instance { timing_spec } )

celltype ::= ( CELLTYPE qstring )
```

```
cell_instance ::=
   ( INSTANCE [ hierarchical_identifier ] )
  | ( INSTANCE * )

character ::=
   alphanumeric
  | escaped_character

cns_def ::=
   path_constraint
  | period_constraint
  | sum_constraint
  | diff_constraint
  | skew_constraint

concat_expression ::= , simple_expression

condelse_def ::= ( CONDELSE iopath_def )

conditional_port_expr ::=
   simple_expression
  | ( conditional_port_expr )
  | unary_operator ( conditional_port_expr )
  | conditional_port_expr binary_operator
     conditional_port_expr

cond_def ::=
  ( COND [ qstring ] conditional_port_expr iopath_def )

constraint_path ::= ( port_instance port_instance )

date ::= ( DATE qstring )

decimal_digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

delay_file ::= ( DELAYFILE sdf_header cell { cell } )

deltype ::=
   absolute_deltype
  | increment_deltype
  | pathpulse_deltype
  | pathpulsepercent_deltype
```

```
delval ::=
   rvalue
  | ( rvalue rvalue )
  | ( rvalue rvalue rvalue )

delval_list ::=
   delval
  | delval delval
  | delval delval delval
  | delval delval delval delval [ delval ] [ delval ]
  | delval delval delval delval delval delval delval
     [ delval ] [ delval ] [ delval ] [ delval ] [ delval ]

del_def ::=
   iopath_def
  | cond_def
  | condelse_def
  | port_def
  | interconnect_def
  | netdelay_def
  | device_def

del_spec ::= ( DELAY deltype { deltype } )

departure_env ::=
  ( DEPARTURE [ port_edge ] port_instance rvalue rvalue
    rvalue rvalue )

design_name ::= ( DESIGN qstring )

device_def ::= ( DEVICE [ port_instance ] delval_list )

diff_constraint ::=
  ( DIFF constraint_path constraint_path value [ value ] )

edge_identifier ::=
   posedge
  | negedge
  | 01
  | 10
  | 0z
  | z1
```

```
  | 1z
  | z0

edge_list ::=
    pos_pair { pos_pair }
  | neg_pair { neg_pair }

equality_operator ::=
    ==
  | !=
  | ===
  | !==

escaped_character ::=
    \ character
  | \ special_character
  | \"

exception ::= ( EXCEPTION cell_instance { cell_instance } )

hchar := . | /

hierarchical_identifier ::= identifier { hchar identifier }

hierarchy_divider ::= ( DIVIDER hchar )

hold_timing_check ::= ( HOLD port_tchk port_tchk value )

identifier ::= character { character }

increment_deltype ::= ( INCREMENT del_def { del_def } )

input_output_path ::= port_instance port_instance

integer ::= decimal_digit { decimal_digit }

interconnect_def ::=
  ( INTERCONNECT port_instance port_instance delval_list )

inversion_operator ::=
    !
  | ~
```

```
iopath_def ::=
  ( IOPATH port_spec port_instance { retain_def } delval_list )

lbl_def ::= ( identifier delval_list )

lbl_spec ::= ( LABEL lbl_type { lbl_type } )

lbl_type :=
    ( INCREMENT lbl_def { lbl_def } )
  | ( ABSOLUTE lbl_def { lbl_def } )

name ::= ( NAME qstring )

neg_pair ::=
  ( negedge signed_real_number [ signed_real_number ] )
  ( posedge signed_real_number [ signed_real_number ] )

net ::=
    scalar_net
  | bus_net

netdelay_def ::= ( NETDELAY net_spec delval_list )

net_instance ::=
    net
  | hierarchical_identifier hier_divider_char net

net_spec ::=
    port_instance
  | net_instance

nochange_timing_check ::=
  ( NOCHANGE port_tchk port_tchk rvalue rvalue )

pathpulsepercent_deltype ::=
  ( PATHPULSEPERCENT [ input_output_path ] value [ value ] )

pathpulse_deltype ::=
  ( PATHPULSE [ input_output_path ] value [ value ] )

path_constraint ::=
  ( PATHCONSTRAINT [ name ] port_instance port_instance
    { port_instance } rvalue rvalue )
```

```
period_constraint ::=
  ( PERIODCONSTRAINT port_instance value [ exception ] )

period_timing_check ::= ( PERIOD port_tchk value )

port ::=
    scalar_port
  | bus_port

port_def ::= ( PORT port_instance delval_list )

port_edge ::= ( edge_identifier port_instance )

port_instance ::=
    port
  | hierarchical_identifier hchar port

port_spec ::=
    port_instance
  | port_edge

port_tchk ::=
    port_spec
  | ( COND [ qstring ] timing_check_condition port_spec )

pos_pair ::=
  ( posedge signed_real_number [ signed_real_number ] )
  ( negedge signed_real_number [ signed_real_number ] )

process ::= ( PROCESS qstring )

program_name ::= ( PROGRAM qstring )

program_version ::= ( VERSION qstring )

qstring ::= " { any_character } "

real_number ::=
    integer
  | integer [ . integer ]
  | integer [ . integer ] e [ sign ] integer
```

```
recovery_timing_check ::=
  ( RECOVERY port_tchk port_tchk value )

recrem_timing_check ::=
    ( RECREM port_tchk port_tchk rvalue rvalue )
  | ( RECREM port_spec port_spec rvalue rvalue
      [ scond ] [ ccond ] )

removal_timing_check ::= ( REMOVAL port_tchk port_tchk value )

retain_def ::= ( RETAIN retval_list )

retval_list ::=
    delval
  | delval delval
  | delval delval delval

rtriple ::=
    signed_real_number : [ signed_real_number ] :
      [ signed_real_number ]
  | [ signed_real_number ] : signed_real_number :
      [ signed_real_number ]
  | [ signed_real_number ] : [ signed_real_number ] :
      signed_real_number

rvalue ::=
    ( [ signed_real_number ] )
  | ( [ rtriple ] )

scalar_constant ::=
    0
  | 'b0
  | 'B0
  | 1'b0
  | 1'B0
  | 1
  | 'b1
  | 'B1
  | 1'b1
  | 1'B1
```

```
scalar_net ::=
    hierarchical_identifier
  | hierarchical_identifier [ integer ]

scalar_node ::=
    scalar_port
  | hierarchical_identifier

scalar_port ::=
    hierarchical_identifier
  | hierarchical_identifier [ integer ]

scond ::= ( SCOND [ qstring ] timing_check_condition )

sdf_header ::=
  sdf_version [ design_name ] [ date ] [ vendor ]
  [ program_name ] [ program_version ] [ hierarchy_divider ]
  [ voltage ] [ process ] [ temperature ] [ time_scale ]

sdf_version ::= ( SDFVERSION qstring )

setuphold_timing_check ::=
    ( SETUPHOLD port_tchk port_tchk rvalue rvalue )
  | ( SETUPHOLD port_spec port_spec rvalue rvalue
      [ scond ] [ ccond ] )

setup_timing_check ::= ( SETUP port_tchk port_tchk value )

sign ::= + | -

signed_real_number ::= [ sign ] real_number

simple_expression ::=
    ( simple_expression )
  | unary_operator ( simple_expression )
  | port
  | unary_operator port
  | scalar_constant
  | unary_operator scalar_constant
  | simple_expression ? simple_expression : simple_expression
  | { simple_expression [ concat_expression ] }
  | { simple_expression { simple_expression
      [ concat_expression ] } }
```

```
skew_constraint ::= ( SKEWCONSTRAINT port_spec value )

skew_timing_check ::= ( SKEW port_tchk port_tchk rvalue )

slack_env ::=
  ( SLACK port_instance rvalue rvalue rvalue rvalue
    [ real_number ] )

special_character ::=
  ! | # | % | & | ‘ | ( | ) | * | + | , | - | . | / | : | ; | < | = | > | ? |
    @ | [ | \ | ] | ^ | ` | { | | | } | ~

sum_constraint ::=
  ( SUM constraint_path constraint_path { constraint_path }
    rvalue [ rvalue ] )

tchk_def ::=
    setup_timing_check
  | hold_timing_check
  | setuphold_timing_check
  | recovery_timing_check
  | removal_timing_check
  | recrem_timing_check
  | skew_timing_check
  | bidirectskew_timing_check
  | width_timing_check
  | period_timing_check
  | nochange_timing_check

tc_spec ::= ( TIMINGCHECK tchk_def { tchk_def } )

temperature ::=
    ( TEMPERATURE rtriple )
  | ( TEMPERATURE signed_real_number )

tenv_def ::=
    arrival_env
  | departure_env
  | slack_env
  | waveform_env
```

```
te_def ::=
    cns_def
  | tenv_def

te_spec ::= ( TIMINGENV te_def { te_def } )

timescale_number ::= 1 | 10 | 100 | 1.0 | 10.0 | 100.0

timescale_unit ::= s | ms | us | ns | ps | fs

time_scale ::= ( TIMESCALE timescale_number timescale_unit )

timing_check_condition ::=
    scalar_node
  | inversion_operator scalar_node
  | scalar_node equality_operator scalar_constant

timing_spec ::=
    del_spec
  | tc_spec
  | lbl_spec
  | te_spec

triple ::=
    real_number : [ real_number ] : [ real_number ]
  | [ real_number ] : real_number : [ real_number ]
  | [ real_number ] : [ real_number ] : real_number

unary_operator ::=
    +
  | -
  | !
  | ~
  | &
  | ~&
  | |
  | ~|
  | ^
  | ^~
  | ~^
```

```
value ::=
    ( [ real_number ] )
  | ( [ triple ] )

vendor ::= ( VENDOR qstring )

voltage ::=
    ( VOLTAGE rtriple )
  | ( VOLTAGE signed_real_number )

waveform_env ::=
  ( WAVEFORM port_instance real_number edge_list )

width_timing_check ::= ( WIDTH port_tchk value )
```

❑